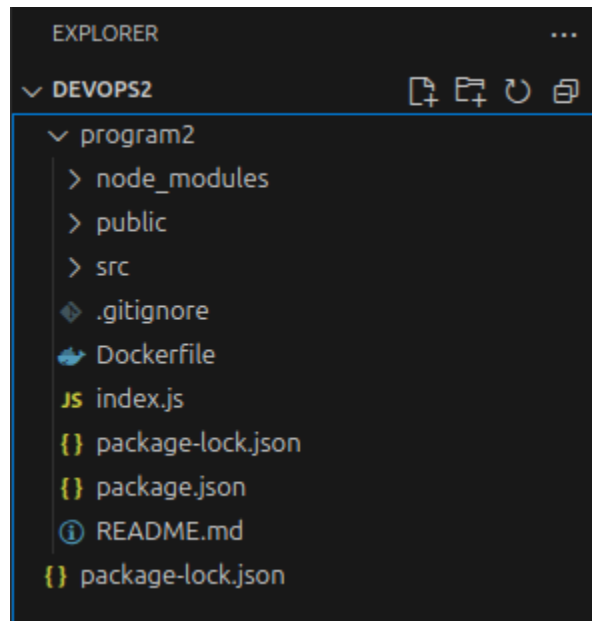


Program 2: Develop a multi stage docker file for container orchestration

Project structure:



Step 1: create a directory
mkdir Devops
cd Devops

Step 2: create a Dockerfile
nano Dockerfile

```
{} package.json  Dockerfile x  JS index.js  {} package-lock.json
program2 > Dockerfile > ...
1  FROM node:20-alpine
2
3  WORKDIR /app
4
5  COPY package*.json ./
6  RUN npm install
7
8  COPY . .
9
10 EXPOSE 3000
11
12 CMD ["node", "index.js"]
13
```

Step 3: create a [index.js](#) file

nano index.js

```
{} package.json  Dockerfile  JS index.js x  {} package-lock.json
program2 > JS index.js > ...
1  const express = require('express');
2  const app = express();
3  const PORT = 3000;
4
5  app.get('/', (req, res) => {
6    res.send('Hello from Express!');
7  });
8
9  app.listen(PORT, () => {
10    console.log(`Server running on port ${PORT}`);
11  });
12
```

Step 4: alter the package.json file

```
{} package.json x Dockerfile JS index.js {} package-lock.json
program2 > {} package.json > ...
1  {
2    "name": "program-2",
3    "version": "1.0.0",
4    "description": "A Node.js app with multi-stage Dockerfile",
5    "main": "dist/index.js",
6    "scripts": {
7      "start": "node dist/index.js",
8      "build": "mkdir -p dist && cp -r src/* dist/"
9    },
10   "author": "Deepika K",
11   "license": "MIT",
12   "dependencies": {
13     "express": "^4.18.2"
14   }
15 }
```

Step 5: docker build -t program2 .

```
harsha@HARSHA:~/Devops2/program2$ docker build -t my-node-app .
[+] Building 46.9s (10/10) FINISHED docker:default
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 159B 0.0s
=> [internal] load metadata for docker.io/library/node:20-alpine 8.1s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/node:20-alpine@sha256:6178e78b972f79c335df281f 30.7s
=> => resolve docker.io/library/node:20-alpine@sha256:6178e78b972f79c335df281f4 0.0s
=> => sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db1 3.80MB / 3.80MB 14.1s
=> => sha256:60e45a9660cfaebbbac9bba98180aa28b3966b7f2462d13 42.75MB / 42.75MB 29.1s
=> => sha256:e74e4ed823e9560b3fe51c0cab47dbfd4b12453604319408 1.26MB / 1.26MB 2.9s
=> => sha256:6178e78b972f79c335df281f4b7674a2d85071aae2af020ffa 7.67kB / 7.67kB 0.0s
=> => sha256:be8d32d651b3e0c9c2b28fdd1d3888408125d703232013cff9 1.72kB / 1.72kB 0.0s
=> => sha256:2b56f2779663b9e1a77b5235dc31f1a81e534ccab1c1b35c 6.42kB / 6.42kB 0.0s
=> => sha256:da04d522c98fe12816b2bcddf8413fca73645f8fa60f287c672f58 444B / 444B 4.2s
=> => extracting sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c988 0.1s
=> => extracting sha256:60e45a9660cfaebbbac9bba98180aa28b3966b7f2462d132c46f51a 1.3s
=> => extracting sha256:e74e4ed823e9560b3fe51c0cab47dbfd4b12453604319408ec587 0.1s
=> => extracting sha256:da04d522c98fe12816b2bcddf8413fca73645f8fa60f287c672f58b 0.0s
```

Step 6: `docker run -p 3000:3000 my-node-app`

```
=> => naming to docker.io/library/my-node-app
harsha@HARSHA:~/Devops2/program2$ docker run -p 3000:3000 my-node-app
Server running on port 3000
```

Step 7: check for the output in the browser

<http://localhost:3000>

